Chapter 3: Database Integrity, Security and Recovery

Instructor: Melaku M.

Target Group: G3 SE

Outline

- Integrity
  - Integrity Concept
  - Integrity Constraints
  - Types of Constraints
- Security
  - Database threats
  - Identification and Authentication
  - Categories of Control
  - Implementation of Security Subsystems
  - Data Encryption

Database integrity

- Database integrity is the process of ensuring that data in a database is accurate, complete, consistent, and valid.
- By following the process, organizations not only ensure the integrity of the data but guarantee they have accurate and correct data in their database.
- Importance of database Integrity:
  - **Data Accuracy:** Ensures that data entered into the database is valid and meaningful.
  - **Consistency:** Helps maintain consistent data across related tables.
  - **Error Prevention:** Prevents invalid data entry, reducing the need for error correction later.

Data Integrity issues

❖Here are some examples of data integrity at risk:

1. An attempt to enter a phone number in the wrong format.
2. A developer accidentally tries to insert the data into the wrong table while transferring data between two databases.
3. An attempt to delete a record in a table, but another table references that record as part of a relationship.
4. A user accidentally tries to enter a phone number into a date field.

Integrity Constraints

❖**Organizations can maintain data integrity(the accuracy and consistency of data within a database) through integrity constraints,** which define the rules and procedures around actions like deletion, insertion, and update of information.

❖Main types of integrity constraints:

❖Physical integrity

- Physical integrity means protecting the accuracy, correctness, and wholeness of data when it is stored and retrieved. This is typically compromised by issues like power outages, storage erosion, hackers targeting database functions, and natural disasters, which prevent accurate data storage and retrieval.

❖Logical integrity

- Logical integrity ensures that data remains unchanged while being used in different ways through relational databases. This approach also aims to protect data from hacking or human error issues but does so differently than physical integrity.
- Logical integrity comes in four different formats:

Cont'd

1. Entity Integrity

- Ensures that each table has a primary key that uniquely identifies each row.

2. Referential Integrity

- Maintains consistency between related tables through foreign keys.
- Database structures are embedded with rules that define how foreign keys are used, which ensures only appropriate data deletion, changes, and amendments can be made. This can prevent data duplication and guarantee data accuracy.

Cont'd

3. Domain Integrity

- Ensures that all values in a column fall within a defined set of valid values.
- This can include data types, formats, and ranges (e.g., dates, numerical ranges).
- A domain is classified by a set of values that a table's columns are allowed to contain, along with constraints and measures that limit the amount, format, and type of data that can be entered.

4. User-Defined Integrity

- Custom rules defined by users to enforce specific business logic.
- This is usually used when other standard integrity processes will not safeguard an organization's data integrity.

What is database security?

- Data is a valuable resource that must be strictly controlled and managed, as with any corporate resource.
- Part or all of the corporate data may have strategic importance and therefore needs to be kept secure and confidential.
- Database security refers to the **range of tools**, **controls** and **measures** designed to establish and preserve database confidentiality, integrity and availability.
  - ✓Mechanisms that protect the database against intentional or accidental threats.
- Security considerations do not only apply to the data held in a database. Breaches of security may affect other parts of the system, which may in turn affect the database.

What is database security?

We consider database security about the following situations:

- ❖Theft and fraudulent.
- ❖Loss of confidentiality or secrecy.
- ❖Loss of data privacy.
- ❖Loss of data integrity.
- ❖Loss of availability of data.

Why database security is a serious concern for an organization.

- By definition, a **data breach** is a failure to maintain the confidentiality of data in a database.
- **Compromised intellectual property:** Your intellectual property—trade secrets, inventions, proprietary practices—can be critical to your ability to maintain a competitive advantage in your market. If that intellectual property is stolen or exposed, your competitive advantage can be difficult .
- **Damage to brand reputation:** Customers or partners might be unwilling to buy your products or services (or do business with your company) if they don't feel they can trust you to protect your data or theirs.
- **Business continuity (or lack thereof):** Some businesses cannot continue to operate until a breach is resolved.
- **Fines or penalties for non-compliance:** The financial impact for failing to comply with global regulations(HIPAA).
- **Costs of repairing breaches and notifying customers**

Database threats

❖**Database threats** are potential **risks** or **vulnerabilities** that can compromise the security, integrity, confidentiality, or availability of a database system.

❖Many software misconfigurations, system flaws. malicious actors, vulnerabilities or patterns of carelessness or misuse can result in breaches.

Most Common Types of Database Threats

1. SQL Injection:

- SQL Injection attacks involve the ***insertion of malicious SQL code*** into a ***database query***. This is often done through web application forms or HTTP requests.
- If developers do not adhere to secure coding practices or if the organization does not conduct regular vulnerability testing, the database becomes susceptible to these attacks. SQL Injection can lead to unauthorized access, data theft, and even database corruption.

Cont'd

2. Malware and Ransomware

- ❖**Malicious software** that can damage or lock the database.
- ❖Encrypting data and demanding ransom for decryption (ransomware).
- ❖Installing backdoors for persistent access.

✓Malware is software that is written specifically to take advantage of vulnerabilities or otherwise cause damage to the database. Malware can arrive via any endpoint device connecting to the database's network.

Cont'd

3. Insider Threats:

❖ **Employees** or **contractors** abusing their access to compromise the database.

✓Stealing sensitive data, Deleting or altering data maliciously.

❖An insider threat is a security threat from any one of three sources with privileged access to the database:

✓A malicious insider who intends to do harm.

✓A negligent insider who makes errors that make the database vulnerable to attack.

✓An infiltrator, an outsider who somehow obtains credentials via a scheme, such as phishing or by gaining access to the credential database itself.

Cont'd

4. Denial of service (DoS and DDoS) attacks

- ❖In a denial of service (DoS) attack, the attacker floods the target server—in this case the database server—with so many requests that the server can no longer fulfill legitimate requests from actual users, and, often, the server becomes unstable or crashes.
- ❖In a distributed denial of service attack (DDoS), the deluge comes from multiple sources, making it more difficult to stop the attack.
- ❖Overloading the database with requests to make it unavailable.
  - ✓Sending a flood of queries to overwhelm the database server.

Cont'd

5. Human error

❖Accidents, weak passwords, password sharing and other unwise or uninformed user behaviors continue to be the cause of nearly half (49%) of all reported data breaches.



Cont'd

6. Exploitation of database software vulnerabilities

- Hackers make their living by finding and targeting vulnerabilities in all kinds of software, including database management software.
- All major commercial database software vendors and opensource database management platforms issue regular security patches to address these vulnerabilities, but failure to apply these patches in a timely fashion can increase your exposure.
- **Outdated Software and Patches:** Using outdated database software can leave systems vulnerable to security issue.

Cont'd

7. Unauthorized Access

- Unauthorized access occurs when individuals gain entry to a database without proper permission.
- This can happen due to weak passwords or default passwords, poor access controls, or even social engineering. Many employees are often granted privileged user access, making insider threats a significant concern for unauthorized access.

Cont'd

8. Excessive Privileges

- Oftentimes, database administrators assign privileges to users based on their job roles and responsibilities. These are rightful employees within an organization; however, abuse of privileges is on the other hand.
- By any chance, the admin may grant permission to the wrong ones. Some users misuse their access rights for unauthorized commands.

Cont'd

9. Physical Security Threats

- ❖Physical access to database servers can lead to compromise or destruction.
- ❖**Examples**:
  - ✓Theft of servers or storage devices, Destruction of hardware due to natural disasters or sabotage.

10. Lack of Auditing and Monitoring

- ❖Without proper logging and monitoring, suspicious activity may go unnoticed.
- ❖**Examples:**
  - ❖Undetected data exfiltration, Lack of traceability for unauthorized actions.

Defining identification & authentication

- Identification is the first step in most online transactions and requires a user to “identify” themselves, usually by providing a name, email address, phone number, or username. This is the process of someone saying that they are a certain person.
- In an online environment, however, it can be difficult to verify that a person is giving a real identity and that they are who they say they are.
- Identities can be verified through providing more information, often a form of government-issued ID. The verification process generally only happens the first time you create an account or access a site. After this, your identity will be authenticated, often by the creation of a password to go along with your username.
- When initially signing up, accessing, or onboarding with a system, service, or company —after your identity has been verified — a form of authentication is set up. This will be required each additional time the service or application is accessed.

Identification and Authentication

❖**Identification** is the act of **identifying/recognizing a user** attempting to access the database. It typically involves presenting a unique identifier, such as a username or user ID.

❖Authentication is the process of **verifying/proofing the identity of a user** or entity. After identification, the system checks credentials like password, biometrics to confirm that the user is who they claim to be.

❖Only after a user has been properly **identified** and **authenticated** can they then be authorized access to systems or privileges.

❖The **authorization** aspect assigns rights and privileges to specific resources.

Common Methods of Authentication:

1. Password-Based Authentication:

- ✓ Users enter a username and password.

2.Multi-Factor Authentication (MFA):

- ✓ Requires two or more verification methods (e.g., a password and a one-time code sent to a mobile device).

3. Biometric Authentication:

- ✓ Uses unique biological characteristics (e.g., fingerprints, facial recognition) to verify identity.

4. Token-Based Authentication:

- ✓ Users receive a token after initial login, which they use for subsequent requests. Tokens can be time-limited and revoked.

5. Certificate-Based Authentication:

- ✓ Uses digital certificates to verify identity, commonly employed in secure communications.

How identification, authentication & authorization are used

✓An example would include the following:

- ❖The user is onboarded into the system by providing identification information.
- ❖The user sets up an authentication factor, such as a password, for future entrance.
- ❖The user returns to login and the system asks for the identification (username) and authentication factor (password).
- ❖The system authenticates the user by verifying that the information is correct and matches what is stored.
- ❖The user is granted access to systems and resources that the admin has authorized.

Best Practices for Identification and Authentication

- Best Practices for Identification and Authentication
- **Strong Password Policies:**
  - Enforce complexity requirements, regular updates, and prevent reuse of old passwords.
  - **Account Lockout Mechanisms:**
  - Implement lockout policies after a certain number of failed login attempts to deter brute-force attacks.
  - **Role-Based Access Control (RBAC):**
  - Assign permissions based on user roles, minimizing the principle of least privilege.
  - **Regular Audits:**
  - Conduct regular reviews of user access logs and authentication methods to identify potential breaches or vulnerabilities.
  - **Secure Communication:**
  - Use secure protocols (e.g., SSL/TLS) to encrypt data transmitted during authentication processes.

Categories of Access control

❖Access control in databases refers to the ***techniques and mechanisms used to regulate and restrict access to data***, ensuring that only authorized users or processes can perform specific actions on the database.

❖ This is a key component of database security, ensuring CIA of data.

❖Database access control models determine how permissions and roles are assigned and enforced.

Main access control techniques used in databases are:

A. Discretionary Access Control (DAC)

- ❖ DAC allows owners or administrators to grant or revoke access.

B. Mandatory Access Control (MAC)

- ❖ MAC assigns security levels or labels to data objects and users, permitting access based on matching levels.

C. Role-Based Access Control (RBAC)

- ❖ RBAC assigns roles with specific permissions based on functions or responsibilities.

D. Attribute-Based Access Control (ABAC)

- ❖ ABAC evaluates attributes and applies rules to determine access levels. These models play a vital role in managing database access effectively.

A. Discretionary Access Control (DAC)

❖DAC allows the owner (or creator) of a database object (e.g., table, view, or record) to determine who can access it and what actions they can perform.

- ✓Based on user identities and permissions.
- ✓Flexible but potentially less secure as permissions can propagate (e.g., a user granted access can share it with others).

❖Implementation in Databases:

- ✓Granting permissions like SELECT, INSERT, UPDATE, DELETE, etc.
- ✓GRANT SELECT, INSERT ON employees TO user1;
- ✓REVOKE INSERT ON employees FROM user1;

A. Discretionary Access Control (DAC)

✓**Advantages:**

- ❖Easy to implement and manage.
- ❖Provides fine-grained control over individual objects.

✓**Disadvantages**:

- ❖Susceptible to privilege abuse (e.g., users sharing access inappropriately).

B. Mandatory Access Control (MAC)

❖MAC enforces access control policies based on fixed rules defined by the system, rather than by the object owner.

❖Permissions are determined by security labels assigned to users and database objects.

- ✓Users and data are classified into levels (e.g., Top Secret, Confidential, Public).
- ✓Users can only access data for which they have the appropriate clearance.

❖**Implementation in Databases:**

- ✓Security labels (or classifications) are applied to both users and data.
- ✓Policies are defined to enforce access rules.

B. Mandatory Access Control (MAC)

❖**Advantages**:

✓Highly secure and difficult to bypass.

✓Suitable for environments with strict security needs (e.g., military or government databases).

❖**Disadvantages:**

✓Complex to implement and manage.

✓Less flexible compared to DAC.

C. Role-Based Access Control (RBAC)

❖RBAC assigns permissions to roles rather than to individual users. Users are then assigned roles, and their access is determined by the roles they hold.

- ✓Roles are created based on job functions (e.g., admin, data_entry, analyst).
- ✓Users inherit permissions through their assigned roles.

❖Implementation in Databases:

- ✓Define roles and assign permissions to roles.
- ✓Assign roles to users.

❖SQL Example (MYSQL):

- ✓CREATE ROLE data_entry;
- ✓GRANT SELECT, INSERT ON employees TO data_entry;
- ✓GRANT data_entry TO user1;

C. Role-Based Access Control (RBAC)

❖**Advantages**:

✓Simplifies management of permissions for large systems.

✓Ensures consistency and aligns with organizational policies.

❖**Disadvantages**:

✓Requires careful planning and role definition.

✓May be less granular than DAC.

D. Attribute-Based Access Control (ABAC)

- ABAC uses attributes (e.g., user attributes, object attributes, environment conditions) to enforce access control policies.
- **Key Features:**
  - Attributes can include user roles, time of access, location, data sensitivity, etc.
  - Policies define conditions under which access is granted.
- **Implementation in Databases:**
  - Often implemented with policy engines or external tools.
  - **Example: Granting access based on department or location.**
- Example Policy:
  - Only users in the "HR" department can view the "employee_salaries" table.

D. Attribute-Based Access Control (ABAC)

❖**Advantages**:

- ✓Highly flexible and dynamic.
- ✓Supports complex, context-aware access decisions.

❖**Disadvantages**:

- ✓Complex to implement and manage.
- ✓Requires a robust policy definition framework.

Best Practices for Database Access Control

- **Principle of Least Privilege:** Grant users the minimum permissions required for their tasks.
- **Separation of Duties:** Divide responsibilities to prevent abuse of privileges.
- **Regular Audits:** Periodically review and update access permissions.
- **Logging and Monitoring:** Log access attempts and monitor for suspicious activities.
- **Use Strong Authentication:** Combine access control with secure identification and authentication mechanisms (e.g., MFA).
- **Encrypt Sensitive Data:** Protect sensitive data even if unauthorized access occurs.
- **Role Management:** Regularly review and refine roles to align with organizational changes.

Implementation of Security Subsystem

- Implementing a security subsystem in a database environment involves several steps and considerations to ensure that data is protected from unauthorized access, data breaches, and other potential threats.
- A comprehensive security subsystem addresses key areas such as authentication, authorization, encryption, auditing, and backup/recovery.
- Structured guideline for implementing a database security subsystem are discussed in the following sections:

Implementation of Security Subsystem

❖Structured guideline for implementing a database security subsystem are:

1. Authentication Subsystem
2. Authorization Subsystem
3. Data Encryption
4. Auditing and Logging
5. User Education and Training
6. Physical Security Subsystem
7. Backup and Recovery Subsystem

1. Authentication Subsystem

❖**Authentication ensures that only authorized users can access the database.**

❖**Define Authentication Methods**:

- User Accounts: Create unique user accounts with strong passwords.
- Password-based authentication: Enforce strong password policies.
- Multi-Factor Authentication (MFA): Combine passwords with an additional factor (e.g., One-Time Passwords, biometrics).

❖**Configure the Database for Authentication**:

- Enforce password complexity and expiration policies, regular updates, and prevent reuse of old passwords. Set account lockout thresholds for failed login attempts.

❖**Secure Credentials**

- Hash and salt passwords. Avoid hardcoding credentials in applications.

❖**Secure Communication:**

- Use secure protocols (e.g., SSL/TLS) to encrypt data transmitted during authentication processes.

2. Authorization Subsystem

- Authorization ensures that users can only access the data and perform actions they are explicitly allowed to.
- Access Control Lists (ACLs): Define ACLs at various levels (database, table, row, column) to grant or deny specific permissions (read, write, update, delete, execute).
- Role-Based Access Control (RBAC): Assign users to roles with predefined permissions. This simplifies administration and reduces the risk of misconfigurations.
- Least Privilege Principle: Grant users only the minimum necessary permissions to perform their job duties.

3. Data Encryption

- Encryption protects sensitive data by converting it into an unreadable format unless decrypted with a key.
- **Data at Rest**: Encrypt data stored on disk using strong encryption algorithms (e.g., AES-256).
- **Data in Transit:** Encrypt data transmitted over the network using protocols like SSL/TLS.
- **Transparent Data Encryption (TDE):** Automatically encrypt and decrypt data transparently to the application.

4. Auditing and Logging

- **Auditing tracks and logs database activities to detect suspicious actions and ensure compliance.**
- **Audit Trail:** Track all database activities (logins, logouts, data access, modifications).
- **Log Analysis:** Regularly analyze audit logs to detect suspicious activity and identify potential security breaches.
- **Enable Database Auditing:** Configure the database to log key events such as login attempts, data modifications, and privilege changes.
- **Intrusion Detection Systems (IDS):** Implement IDS to monitor database traffic for malicious activity.

5. User Education and Training

- Security Awareness Training: Train users on security best practices, including recognizing phishing attacks and secure password usage.
- Incident Response Training: Prepare staff to respond effectively to security incidents.

6. Physical Security Subsystem

- Physical security ensures the database server is protected from unauthorized physical access.
- **Restrict Access to Data Centers:** Use biometric authentication, access cards, or PIN codes.
- **Protect Hardware:** Secure servers in locked racks, Use surveillance systems to monitor data centers.
- **Plan for Disaster Recovery:** Implement redundant systems and power supplies.

7. Backup and Recovery Subsystem

- Backup and recovery mechanisms ensure data availability and integrity in case of failure or attack.
- Schedule Regular Backups
- Encrypt Backups: Protect backups with strong encryption to prevent unauthorized access.
- Test Recovery Procedures: Regularly test backup restoration to verify data integrity and minimize downtime.
- Store Backups Securely: Use offsite or cloud-based storage for redundancy. Implement access controls for backup files.